File system forensics domain

© A.Atzeni (Politecnico di Torino, 2024)

File System Forensics

- **this domain examines the organization and structure of file systems to retrieve hidden or deleted data**
- **key points**
  - Slack Space Analysis
    - unused portions of disk sectors might contain residual data.
  - File Carving
    - deleted files extraction by analyzing residual data clusters
  - "Registry" Analysis, OS config analysis
    - recovering system/user activities (Windows Registry, /etc , ~/.config, ~/.bashrc, ...)
- **relevant Tools**
  - Low-level (*stat, istat, debugfs*)
  - High-level (*FTK Imager* and *Autopsy*)

© A.Atzeni (Politecnico di Torino, 2024)

Meta-data

- **data about... data**
  - e.g. EXIF, ODF, DOCX, ...
- **useful to augment knowledge on file**
- **useful to reveal hidden information**
- **useful to correlate various data**
- **meta-data are not “trusted-by-definition”**

Meta-data example: file system

- **the file system maintains a number of information about file contents**
  - file name, ownership and permission
  - specific data-units allocated to file
  - size of the file
  - time stamps for the file
  - recovery data (e.g. journaling)
  - ...
- **information type and accuracy can vary (very much) depending on file system**
  - FAT32, NTFS, ext2, ext3, ext4, …

Meta-data example: ODF

- **Open document format (ODF) is an OpenOffice.org file format**
  - license-free
  - open
  - XML based
- **if decompressed (zip file) meta-data are in a separate file (*meta.xml*)**

meta.xml sample

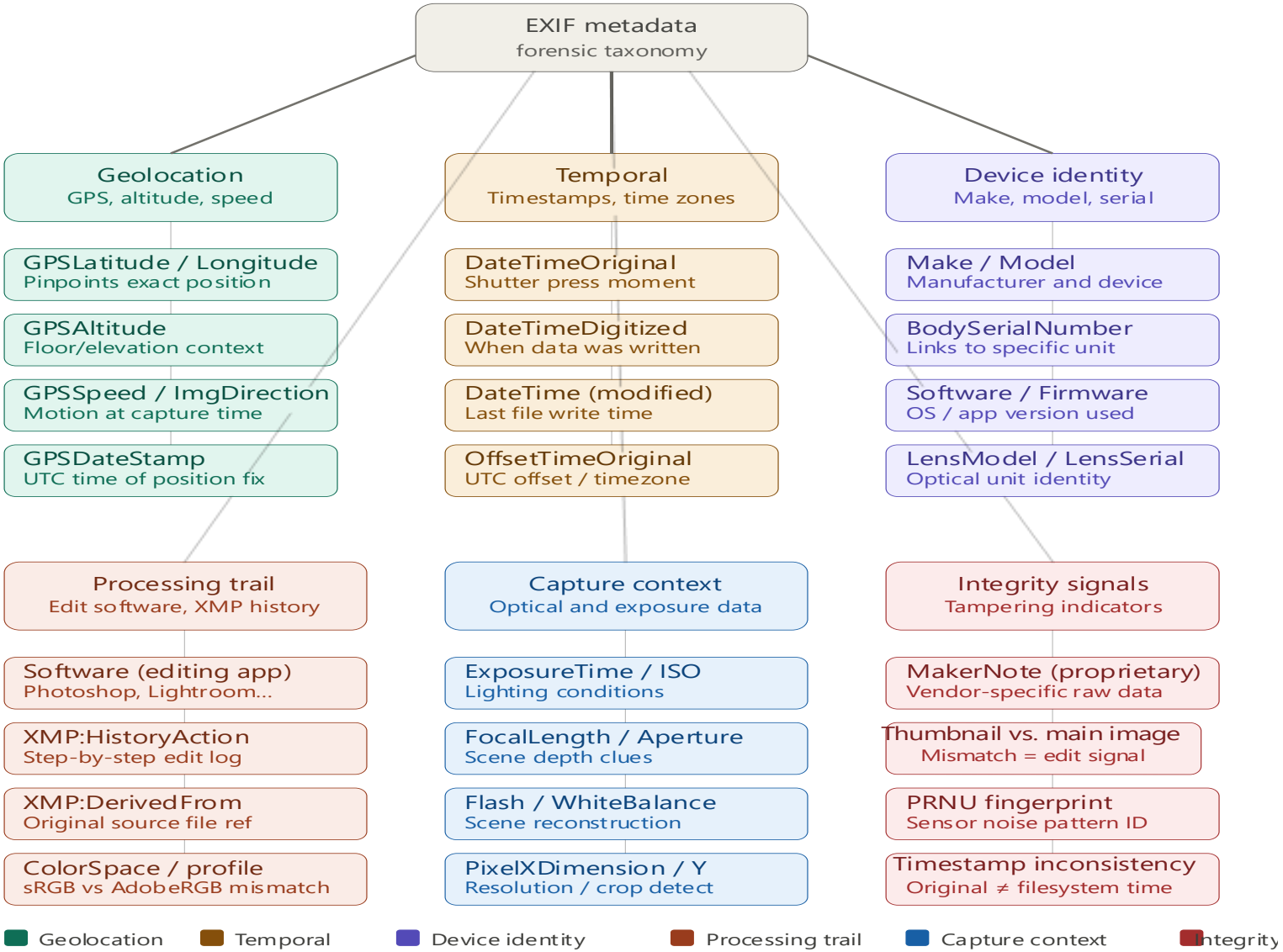
```
<dc:title>Pippo.odt document</dc:title>
<dc:subject>A test of metadata OpenOffice management</dc:subject>
<meta:initial-creator>Sh</meta:initial-creator>
<meta:creation-date>2009-09-26T11:22:39</meta:creation-date>
<dc:creator>Sh</dc:creator>
```

Meta-data example: JPEG

- **the most popular format is the Exchangeable Image File (EXIF) one**

Make **NIKON CORPORATION**
Model **NIKON D80**
Xresolution **300.00**
Yresolution **300.00**
ResolutionUnit **Inch**
Software **Ver.1.01**
ExifVersion **Exif Version 2.21**
FocalLenghtIn35mmFilm **27**
ISOSpeedRatings **125**
Orientation **top – left**
DateTimeOriginal **2009:02:17 12:13:09**
PixelXDimension **3872**
PixelYDimensio **2592**
...

EXIF cybersecurity implications



exiftool

- **platform independent PERL library**
- **command line tool**
  - *exiftool /media/sdb1/IMG_1.jpg*
- **read, write, edit meta information**
  - in many different formats
  - for many different products
    - https://exiftool.org/#supported

exiftool support

exiftool.org/#supported

File Type	Support	Description	EXIF	IPTC	XMP	ICC[1]	C2PA	Other
IDML	R	Adobe InDesign Markup Language (ZIP/XML-based)	-	-	-	-	-	R XML ZIP
IIQ	R/W	Phase One Intelligent Image Quality RAW (TIFF-based)	R/W/C	R/W/C	R/W/C	R/W/C	R/D	R/W PhaseOne
IND, INDD, INDT	R/W	Adobe InDesign Document/Template	-	-	R/W/C	-	-	-
INSP	R/W	Insta360 Picture (JPEG-based)	R/W/C	R/W/C	R/W/C	R/W/C	R/D	Supported JPEG Meta Information
INSV	R	Insta360 Video (QuickTime-based)	-	-	R	-	R	R QuickTime
INX	R	Adobe InDesign Interchange (XML-based)	-	-	R	-	-	-
ISO	R	ISO 9660 disk image	-	-	-	-	-	R ISO
ITC	R	iTunes Cover Flow artwork	-	-	-	-	-	R ITC
J2C, J2K, JPC	R	JPEG 2000 codestream	R[3]	R[3]	R	R	-	R Jpeg2000 Photoshop[3]
JP2, JPF, JPM, JPX, JPH	R/W	JPEG 2000 image [Compound/Extended/High-throughput]	R/W/C[3]	R/W/C[3]	R/W/C	R	-	R/W/C Jpeg2000, R Photoshop[3]
JPEG, JPG, JPE	R/W	Joint Photographic Experts Group image	R/W/C	R/W/C	R/W/C	R/W/C	R/D	Supported JPEG Meta Information
JSON	R	JavaScript Object Notation	-	-	-	-	-	R JSON
JXL	R/W	JPEG XL (codestream and ISO BMFF) (Jpeg2000-based)	R/W/C	-	R/W/C	-	-	-
K25	R	Kodak DC25 RAW (TIFF-based)	R	R	R	R	R	-
KDC	R	Kodak Digital Camera RAW (TIFF-based)	R	R	R	R	R	R Kodak
KEY, KTH	R	Apple iWork '09 Keynote presentation/Theme	-	-	-	-	-	R XML ZIP
LA	R	Lossless Audio (RIFF-based)	R[3]	-	R	-	R	R RIFF
LFP, LFR	R	Lytro Light Field Picture	-	-	-	-	-	R Lytro
LIF	R	Leica Image File	-	-	-	-	-	R LIF
LNK	R	Microsoft Shell Link (Windows shortcut)	-	-	-	-	-	R LNK
LRV	R/W	Low-Resolution Video (QuickTime-based)	R/W[3]	R/W[3]	R/W/C	-	-	R/W/C QuickTime
M2TS, MTS, M2T, TS	R	MPEG-2 Transport Stream (used for AVCHD video)	-	-	-	-	-	R M2TS H264 MISB
M4A, M4B, M4P, M4V	R/W	MPEG-4 Audio/Video (QuickTime-based)	R/W[3]	R/W[3]	R/W/C	-	R/D	R/W/C QuickTime
MACOS	R	MacOS "._" sidecar file (may have any extension)	-	-	-	-	-	R XAttr RSRC
MAX	R	3D Studio MAX (FPX-like)	-	-	R	R	-	R FlashPix
MEF	R/W	Mamiya (RAW) Electronic Format (TIFF-based)	R/W/C	R/W/C	R/W/C	R/W/C	R/D	-
MIE	R/W/C	Meta Information Encapsulation (MIE specification)	R/W/C	R/W/C	R/W/C	R/W/C	R/D	R/W/C MIE
MIFF, MIF	R	Magick Image File Format	R	R	R	R	-	R MIFF Photoshop
MKA, MKV, MKS	R	Matroska Audio/Video/Subtitle	-	-	-	-	-	R Matroska
MOBI, AZW, AZW3	R	Mobipocket electronic book (Palm-based)	-	-	-	-	-	R Palm MOBI
MODD	R	Sony Picture Motion metadata (XML PLIST-based)	-	-	-	-	-	R PLIST
MOI	R	MOD Information file	-	-	-	-	-	R MOI
MOS	R/W	Creo Leaf Mosaic (TIFF-based)	R/W/C	R/W/C	R/W/C	R/W/C	R/D	R Leaf
MOV, QT	R/W	Apple QuickTime Movie	R/W[3]	R/W[3]	R/W/C	-	R/D	R/W/C QuickTime
MP3	R	MPEG-1 layer 3 audio	-	-	-	-	R	R MPEG ID3 Lyrics3 APE
MP4	R/W	Motion Picture Experts Group version 4 (QuickTime-based)	R/W[3]	R/W[3]	R/W/C	-	R/D	R/W/C QuickTime
MPC	R	Musepack Audio	-	-	-	-	R	R MPC ID3 Lyrics3 APE
MPEG, MPG, M2V	R	Motion Picture Experts Group version 1 or 2	-	-	-	-	R	R MPEG ID3 Lyrics3
MPO	R/W	Extended Multi-Picture format (JPEG with MPF extensions)	R/W/C	R/W/C	R/W/C	R/W/C	R/D	Supported JPEG Meta Information
MQV	R/W	Sony Mobile QuickTime Video	R/W[3]	R/W[3]	R/W/C	-	R/D	R/W/C QuickTime
MRW	R/W	Minolta RAW	R/W/C	R/W/C	R/W/C	R/W/C	R/D	R/W MinoltaRaw Minolta
MRC	R	Medical Research Council	-	-	-	-	-	R MRC
MXF	R	Material Exchange Format	-	-	-	-	-	R MXF
NEF	R/W	Nikon (RAW) Electronic Format (TIFF-based)	R/W/C	R/W/C	R/W/C	R/W/C	R/D	R/W Nikon NikonCapture
NKA	R	Nikon NX Studio Adjustments	-	-	-	-	-	R XML
NKSC	R/W	Nikon Sidecar (XMP-based)	-	-	R/W/C	-	-	-
NMBTEMPLATE	R	Apple iWork '09 Numbers Template	-	-	-	-	-	R XML ZIP
NRW	R/W	Nikon RAW (2) (TIFF-based)	R/W/C	R/W/C	R/W/C	R/W/C	R/D	R/W Nikon NikonCapture
NXD	R	Nikon Capture NX-D adjustments (XMP-based)	-	-	R	-	-	-
NUMBERS	R	Apple iWork '09 Numbers spreadsheet	-	-	-	-	-	R XML ZIP
O	R	Unix compiled code Object	-	-	-	-	-	R EXE
ODB, ODC, ODF, ODG, ODI, ODP, ODS, ODT	R	Open Document Database/Chart/Formula/Graphics/ Image/Presentation/Spreadsheet/Text (ZIP/XML-based)	-	-	-	-	-	R XML ZIP
OFR	R	OptimFROG audio (RIFF-based)	R[3]	-	R	-	R	R RIFF
OGG, OGV	R	Ogg bitstream container	-	-	-	-	R	R FLAC ID3 Lyrics3 Theora Vorbis

Forensic contribution of EXIF meta-data

- **Timeline reconstruction**
  - timestamps embedded in files allow investigators to reconstruct a precise chronology of events
- **Attribution and identity**
  - metadata often embeds author's username, organization name registered in the software license, revision history, ...
- **Geolocation (GPS coordinates - also altitude)**
- **Source device identification**
  - e.g. sensor noise patterns (PRNU – Photo Response non uniformity) link to a specific camera unit
- **Cross-source correlation**
  - phone call logs, cloud synchronization logs, GPS tracks, etc
- **Integrity**
  - Different apps (e.g. messaging such WA, Signal, Instagram, …) alter EXIF information (CF requires awareness of transmission mean)

Other Metadata Sources (examples)

- **Microsoft Office / OOXML documents (.docx, .xlsx, .pptx)**
  - e.g. author name, last editor, company name, total editing time, number of revisions, and a revision history with timestamps
- **PDF metadata**
  - creator application, creation and modification dates, author, the PDF producer (which can reveal if a document was converted from another format or "printed to PDF" to hide its origin), embedded XMP (Extensible Metadata Platform) metadata even after superficial editing
- **Email metadata**
  - Chain of server relay stamps, originating IP addresses, message IDs, client software signature, …
- **File system metadata**

File System

- **a file is the smallest logical unit from an user perspective that can be stored**
  - can be stored in bytes, lines, records, …
- **logical files are mapped into “physical” entities (computer RAM, HD, the Cloud, …) by the Operating System**
  - memory addresses, disk sectors, remote resources, …
- **file systems define the organization and structure of files on a computing device**
- **through the OS, a file system define the rules to read, write and maintain the data**

File Attributes

- **name**
  - a mnemonic (human) id for reference
  - DOS legacy: 8 char+3char for the extension (no modern OS still have this limitation, but some names still have)
- **type**
  - categorize the file to indicate how should be manipulated
  - “magic number” (few bytes) usually at the file start
    - Note: WinOS often rely on extension to trigger execution/associate applications

File Attributes

- **protection**
  - access control information
  - differ depending on OS/FS combination
    - e.g. owner and group (unix-like)
    - read, write, execute (unix-like)
- **location**
- **size**
- **and some other**

File system formatting

- **the operation that prepare a mass storage for data storage, configuring it with specific file system structures**
  - erases existing information
    - full formatting (slow – erases all sectors with zeros/mark bad ones)
    - quick formatting (eras file system tables)
  - creates one/many partitions
    - each partition can be formatted separately (different logical volumes – primary, extended, logical)
  - selects specific file systems (e.g. NTFS for windows, APFS for MacOSX, ext4 for Linux)
  - creates foundational structures
    - Root directory, FAT/Inode table, superblock/Boot sector

Digital storage structures

- **Master Boot Record (MBR - old hard drives)**
  - very first sector of the disk (sector 0 – 512 bytes)
  - 446 bytes master boot code
  - 64 bytes partition table – each partition 16 bytes
    - max 4 primary partition
  - boot signature (2 bytes – 0x55AA)
  - fragile (if sector 0 corrupted) and maximum 2TB (32-bit addr.)
- **GUID Partition Table (GPT) - modern standard UEFI**
  - fixed portion

Digital storage structures (II)

- **GUID Partition Table (GPT) - modern standard Unified Extensible Firmware Interface (UEFI)**
  - fixed portion at the beginning
  - Sector 0 contains a "fake" MBR (a.k.a. *Protective MBR*)
    - prevents old non-GPT app from seeing the disk as empty
  - *Primary GPT Header* contains *disk signature*, *table location*, and a *CRC32 checksum* to verify that data is not corrupted.
  - by default supports up to 128 partitions (can be increased).
  - *Backup GPT Table* identical copy of the primary in the last sectors of the disk.
    - If primary is corrupted, the system restores it from the end
  - GPT Header (LBA 1) defines the usable space on the disk and the integrity of the partition table itself

Digital storage structures (III)

- **Volume Boot Record (VBR)**
  - every partition has its own VBR in the first sector of the partition
  - specific File System data (NTFS, FAT32) needed to load the OS kernel from that specific volume.
- **every partition contains a structure to catalog files**

FAT example

- **when formatting the HD**
  - the Boot Record is created
    - OS name and version
    - disk physical characteristics (bytes per sectors, sectors per cluster, root directory entry)
  - the Master File Table is created (2 times!)
    - info on clusters (available, allocated, damaged, containing OS files)
  - the Directory table is created
    - top level folder file and directory information

FAT example

- **File Allocation Table**
  - i.e. where the OS records the files' position
- **FAT close to the begin of the volume**
  - specified in the boot sector
  - 2 copies (optional, for redundancy)

Boot sector	**Reserved**	**FAT 1**	**FAT 2**	**Root Folder**	**Other folders and files**

FAT example

directory entry

file.dat	other … other	29	3450
file name	other attributes (e.g. RO, hidden)	file start cluster number	file size

File allocation table (subset)

…	00	00	EOF	30	33	32	EOF	EOF	00
25	26	27	28	29	30	31	32	33	34

FAT pros and cons

- portability (available to multiple operating systems)
- migration (easy switch to richer FS, like NTFS)
- fast on small volumes (some GB), due to light "infrastructure (few metadata, no file index, ...)
- no integrated advanced features (on-the-fly compression, user quotas)
- slow for high numer of files (linked-list structure, fragmentation, no index)
- no security at all (encryption, access control lists, ...)

FAT references

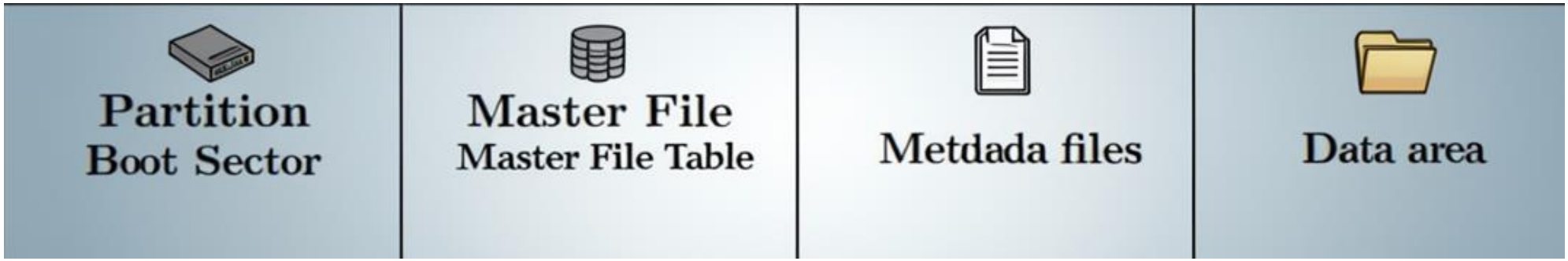
- **https://download.microsoft.com/download/1/6/1/161ba512-40e2-4cc9-843a-923143f3456c/fatgen103.doc**
  - Microsoft Extensible Firmware Initiative FAT32 File System Specification - FAT: General Overview of On-Disk Format
- **https://forensics.wiki/fat/**
  - e.g. recovery in FAT accomplished looking for entries that begin with a sigma 0xe5

NTFS example (ack to Demaria – former CFCCA student)

- **primary file system for Windows**
- **it was designed to replace FAT and HPFS (High Performance File System from IBM – OS/2)**
- **offering robust security and scalability**
  - Master File Table as relational-like metadata
  - attribute-based file model
    - files as collection of typed attributes, not just byte streams
  - write-ahead logging (in *$LogFile*)
    - before modifying metadata log the intent
    - replay commited transitions and roll-back of incomplete modifications, to prevent partial updates (classical FAT issue)

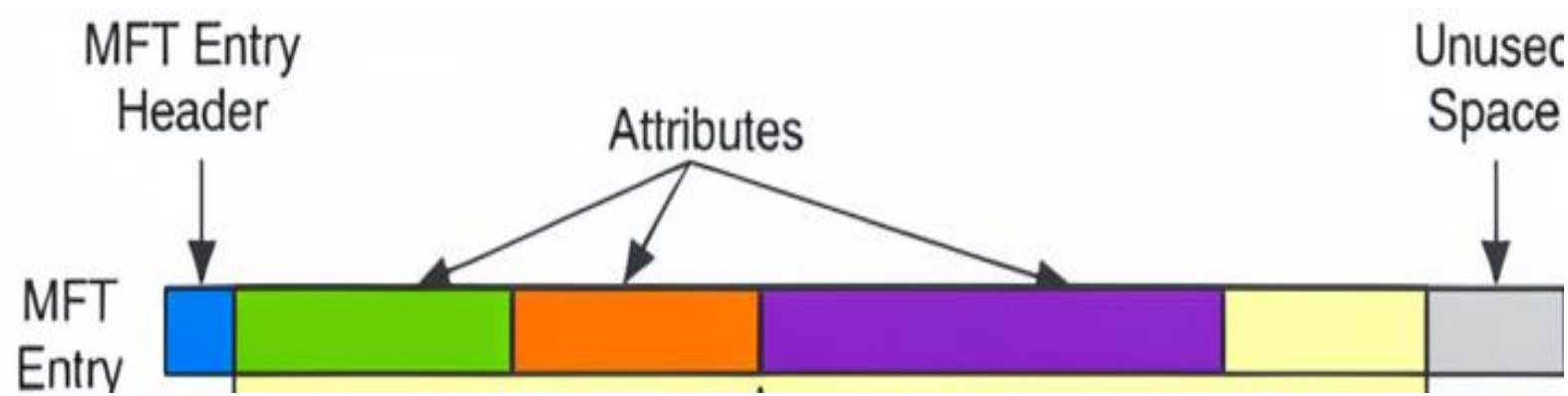
NTFS structure overview

- **Partition Boot Sector (PBS)**
  - boot information and MFT location
- **Master File Table (MFT)**
  - Core element storing metadata for all files and directories
- **Metadata Files**
  - System files for NTFS management
- **Data Area**
  - Actual file and folder data stored in clusters

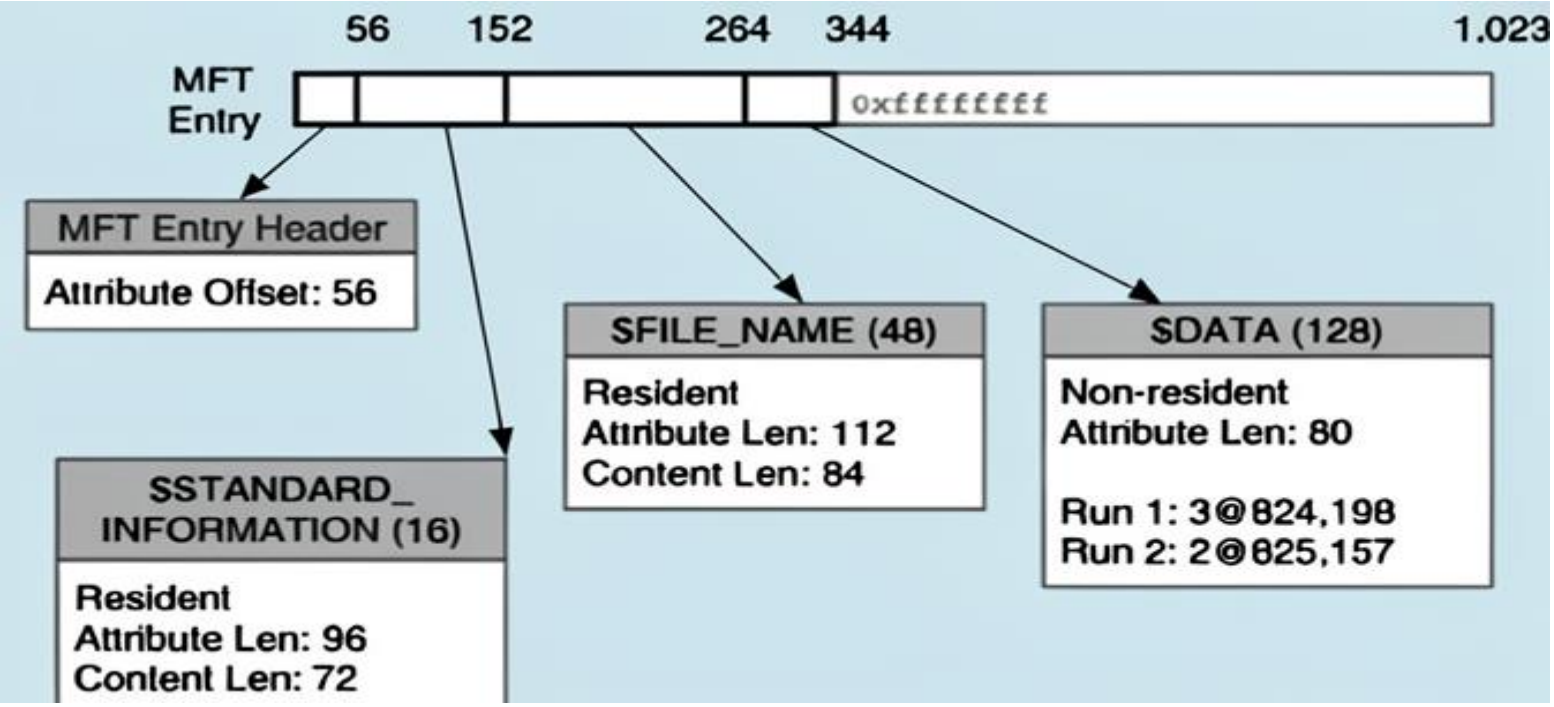


NTFS –MFT

- **1024 bytes in size, two components**
- **Header (fixed size of 42 bytes)**
  - essential metadata about the file or directory
- **Attributes (variable size)**
  - store different types of metadata
  - Resident vs. Non-Resident: Data may be stored inside(resident) or outside the MFT(non resident) entry itself

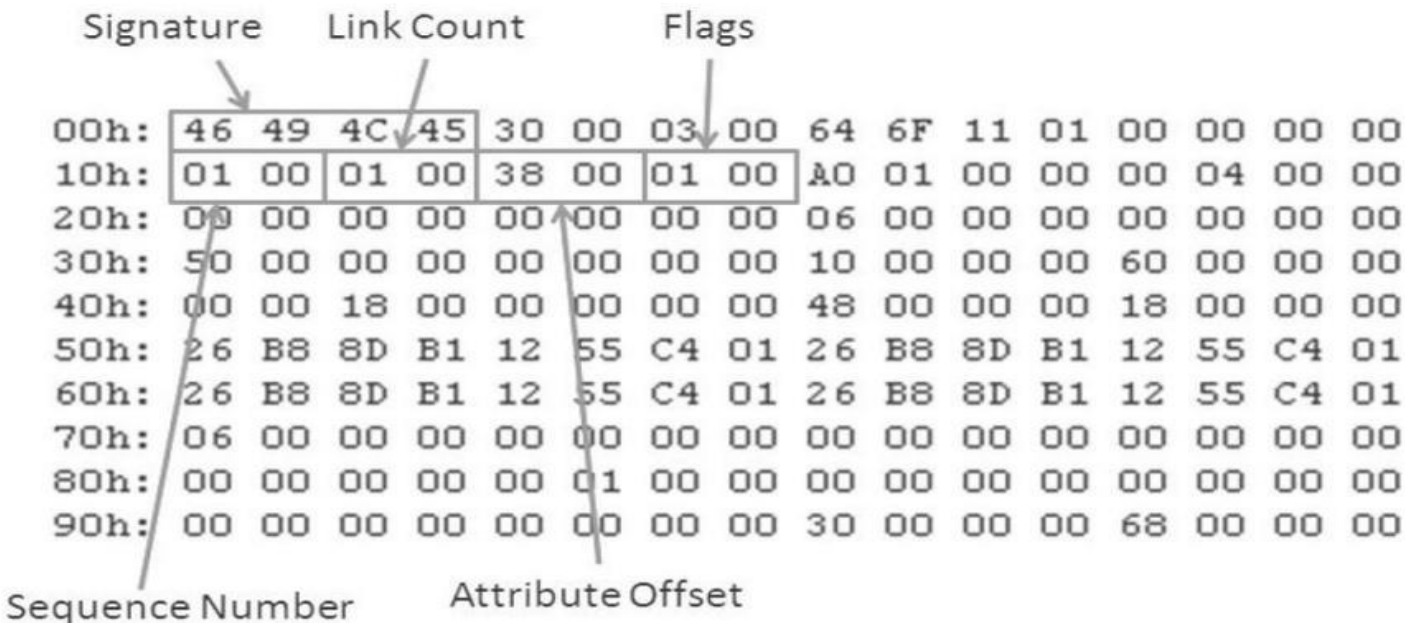


NTFS –MFT example



- The example above shows
  - `$STANDARD_INFORMATION` (resident), `$FILE_NAME` (resident)
  - `$DATA` (non-resident, two data runs at clusters 824,198 and 825,157).

NTFS –MFT header field



Signature	ASCII string "FILE" — identifies a valid file record
Sequence Number	Tracks how many times this MFT record has been reused
Hard Link Count	Number of directory entries referencing this record
First Attribute Offset	Offset where the first attribute begins in the entry
Flags	Record status (in use or not) "Not in use" = deleted but not yet overwritten

MFT entry attributes

- a sequential collection of stream descriptors (attributes)
- packed into one/more MFT records (containing the attributes list)
- each attribute characterised by
  - an attrib. type (a fixed-size integer in file $AttrDef)
  - an optional attribute name (for example, used as the name of an alternate data stream),
  - a value represented as a sequence of bytes.

Example attribute	Description
$DATA	File data (always present)
$FILE_NAME	Stores the file name (always present)
$STANDARD_INFORMATION	File times, quota data (always present)
$SECURITY_DESCRIPTOR	ACL and SIDs
$ATTRIBUTE_LIST	Overflow attributes to additional MFT records
$BITMAP	Maps MFT records in/out of use (directories)

meta-data files

- **specific files that organise the file system**

#	File	Forensic Relevance
0	`$MFT`	The MFT itself
1	`$MFTMirr`	Backup of first 4 MFT records
2	`$LogFile`	Transaction log - reconstructs deletions/modifications after crashes
3	`$Volume`	Volume label, version, flags
6	`$Bitmap`	Cluster usage map - locates unallocated clusters with deleted file remnants
8	`$BadClus`	Bad sectors - potential hiding spot for malicious data
9	$Secure	ACL – relevant to investigate unhauthorized access

File Deletion

- **NTFS marks it as deleted**
  - but the actual data on disk remains untouched
- **The OS**
  - marks MFT entries as *not in use*
  - removes the entry from the parent directory's MFT record
  - Updates MFT bitmap - records available for reuse
  - Updates `$Bitmap` - clusters marked free
- **then, files still present on disk but no longer linked to any directory entry (*not in use*)**
- **scanning raw disk or image for file signatures (headers/footers) without relying on file system metadata allows for recovery**

File Analysis - useful commands

- ***(linux)Stat* (system call and command line utility)**
  - File size, ownership (UID, GID), permissions (mode bits), timestamps (atime - last access, mtime - last modification, ctime - metadata change), Inode number, Link count, device ID
  - useful for timeline reconstrution (file tampering detection, attacker activity reconstruction, events correlation)
- ***(Windows) Get-Item (*a PowerShell cmdlet that retrieves a .NET object from a provider)**
  - File name, file size, attributes, timestamps, CreationTime, LastWriteTime, LastAccessTime
- ***Fsutil***
  - exposes low-level NTFS structure
- ***Istat***
  - Physical/low-level file system structures (bypassing the OS)

Forensic copy

- **normal commands preserve the file content…**
- **…but alter the file attributes (meta-data)**
  - e.g. creation data
- **required bit-per-bit copy to avoid any modification**
  - “data dump” (*dd)* copy/convert bit-per-bit
    - *dd if=<inputfile> <of=outputfile>*
  - variants (e.g. *dcfldd, dc3dd*) with CF addedd features
    - e.g. on the fly hashing (md5, sha-1, sha-256, sha-512), pattern wiping, progress report

© A.Atzeni (Politecnico di Torino, 2024)

Forensic copy

- **clone one hard drive onto another:**
  - *dd if=/dev/sda of=/dev/sdb*
- **clone a hard drive to an image file**
  - *dd if=/dev/hda of=/image.img*
- **clone a hard drive to a zip image in 100Mb blocks**
  - *dd if=/dev/hda bs=100M | gzip -c > /image.img*
- **wipe an hard drive with binary 0s**
  - *dcfldd pattern=00 vf=/dev/hdb*
- **write a binary image and calculate hash**
  - *dc3dd if=/var/log/messages of=/tmp/dc3dd hash=sha512*

© A.Atzeni (Politecnico di Torino, 2024)

File identification

- **extension is not a reliable source**
  - literally anyone can change them
- **check the metadata**
  - where available
- **check first bytes of the file can act as signature**
  - https://en.wikipedia.org/wiki/List_of_file_signatures
  - compare the signature with hex dump of the file

Slack space (I)

- **data starting from the end of the file written and up to the end of the sectors designated to the file**
- **i.e. leftover space when a file do not fill exactly a sector multiple size**
  - the difference between logical (bytes) and physical (sectors) file size
- **sectors have fixed dimension**
  - e.g. 512 bytes
- **file do not have such fixed dimension**
  - e.g. 392 bytes
  - this file will result in 120 bytes of slack space

Slack space (II)

- **Slack space is a "dynamic" entity**
- **Example:**
  - file1 : 392 Bytes, sector : 512 Bytes, file2 : 192 Bytes
  - file1 is created
  - …(other operations on the FS)
  - file1 is erased
  - what's the final result?
    - (ans: the final 200 bytes of the original file will survive in the FS)

Recovery process

- **Analysis of the file system structures**
  - e.g. Master File Table (MFT) to store metadata for all files (like type of data and file names)
  - presence of metadata, such as timestamps, enables creating timelines, essential in criminal cases to determine user actions and verify evidence integrity
  - OSs organizes metadata into system files that assist with permissions, integrity, and file tracking, supporting security and data reliability crucial in legal contexts.
  - Data can be recovered by "deleted"/orphan files/file signatures

hexdump, hexedit, ghex

- **hexdump: command line utility to display a file in a specified format (default hex)**
  - *hexdump <filename>*
- **hexedit: hexadecimal command line file viewer and editor**
  - *hexedit <filename>*
- **ghex: graphic (GNOME) hexadecimal editor**
  - allows user to load data from any file, view and edit it either in hex or ASCII format
  - *ghex <filename>*

Foremost

- **a command line tool that “curves” data from disk images…**
- **…i.e. inspect content of an image looking for erased files**
- **Identify file types on the base of file signature and metadata**
  - *foremost -t jpg,gif –I image.dd –o outdir*
    - look for .jpg and .gif data
    - in the image.dd disk image
    - and put the recovered files in outdir

Photorec

- **Recovering deleted files from a variety of storage media, including hard drives, CD-ROMs, USB flash drives, and memory cards**
  - bypass the file system to scan directly on the storage medium
    - very effective on damaged/formatted storage
  - uses file carving/signature matching techniques (based on header and footer patterns)
  - e.g.
    - *Sudo photorec*
    - choose storage device (like /dev/sda, or HDD image file)
    - choose recovery operation

Digital Forensics and genAI

*Special thanks to **Alessia Angelone**, whose work significantly inspired these slides*

Large Language Models (LLMs)

- **advanced AI systems designed to understand and generate human-like output (text, images, …)**
- **sophisticated neural networks that enable to process and produce coherent and contextually relevant language**
  - contain billions (or even trillions) of parameters
    - learned during training and represent the relationships and patterns within the data
    - allows these models to capture intricate details of language and generalize effectively across a wide range of contexts
  - trained on extensive datasets
  - transformer architectures and self-attention mechanisms
    - enabling the model to compute the relationships between words/tokens regardless of their distance

Using LLMs in investigative processes (I)

- **Enhance efficiency/effectiveness of various phases of an investigative process**
  - support in analyzing textual and visual data, automating tasks, and generating actionable insights
- **Log/data analysis for early detection**
    - parsing of large volumes of unstructured or semi-structured logs and interpretation of complex linguistic clues
    - suspicious activity flag
    - detection of potential security breaches, unauthorized access, IoC

Using LLMs in investigative processes (II)

- **LLMs can streamline the the collection phase identifying and organizing critically evidence, both textual and visual**
  - GPT-4 or newer excels in analyzing textual content, drafting reports, interpreting written evidence
  - special-purpose LLMs like **VisionLLM** combine natural language processing (NLP) with computer vision to analyze and categorize visual data such as photos and videos
    - can identify objects, faces, or text within images and classify them into relevant categories for further investigation

Using LLMs in investigative processes (III)

- **LLMs can automate and optimize tasks in the acquisition and preservation phases**
  - generate code for securely acquiring/preserving evidence
  - Support creation of automated workflows for extracting and storing data from devices or cloud services.
  - e.g. *AutoGen*, a framework that enables automation using AI agents, can work alongside LLMs to perform repetitive tasks such as data extraction, file categorization, or integrity checks
  - reduces the need for manual intervention, ensuring faster and more accurate acquisition processes

Using LLMs in investigative processes (IV)

- **LLMs can recover relevant data and conduct keyword-based searches with significant precision and efficiency**
  - retrieving data from diverse sources, including damaged or fragmented datasets
  - keyword-based searches with contextual understanding, uncovering relevant data even if the exact keyword isn't explicitly present
  - seamlessly connecting with forensic suites for automated analysis.
  - acting as an intelligent interface for querying databases and visualizing results

Using LLMs in investigative processes (V)

- **LLMs can support in deriving meaning and connecting disparate pieces of information**
  - process large datasets and establish connections between seemingly unrelated data points
  - identify trends, anomalies, or relationships that may not be immediately apparen
  - support RAG (Retrieval-Augmented Generation), i.e. combination of LLMs with external knowledge sources to improve the accuracy and relevance of generated insights
    - e.g. investigators can query a corpus of evidence, and RAG systems will retrieve and analyze the most pertinent documents for review

Using LLMs in investigative processes (VI)

- **LLMs can aid creation of documentation of findings, often adhering to established standards**
  - draft comprehensive and well-structured final reports
  - create summaries
  - detailed evidence analyses
  - conclusions formatted for specific audiences
    - legal, managerial, technical...
    - according to international standard
      - e.g., ISO/IEC 27043:2015

Risks using LLMs (I)

- **performance of LLMs heavily depends on the quality and specificity of their training data**
  - general-purpose LLMs may exhibit biases or inaccuracies when applied to forensic scenarios due to a lack of domain-specific training
  - to improve effectiveness, models should be fine-tuned with forensic-specific datasets,
    - digital logs, legal documents, cybercrime case studies
    - to ensure relevance and reduce bias

Risks using LLMs (II)

- **Can't replicate Investigators' Intuition**
  - investigators often rely on subtle contextual cues and years of expertise to make decisions that LLMs may not yet fully emulate
  - effective use of LLMs requires a partnership between human expertise and machine intelligence, emphasizing the importance of human oversight
- **Balance the benefits of automation and efficiency with the financial and resource demands of implementation**
  - developing, training, and deploying LLMs for forensic applications can be resource-intensive requiring significant computational power and expertise
  - fine-tuning models for specific domains further increases costs

Risks using LLMs (III)

- **Censorship and access to information**
  - LLMs often require extensive datasets, which can lead to ethical concerns regarding data acquisition and usage
  - investigative scenarios may involve sensitive, private, or classified information, raising questions about data ownership and privacy
  - strict ethical guidelines and governance structures are necessary to prevent misuse or overreach in data access
- **False or misleading Information**
  - generation of false, misleading, or inaccurate information,
  - determining accountability for errors becomes complex when AI-generated insights influence investigative decisions
  - legal frameworks and ethical standards must address liability and ensure transparency in AI-assisted investigations